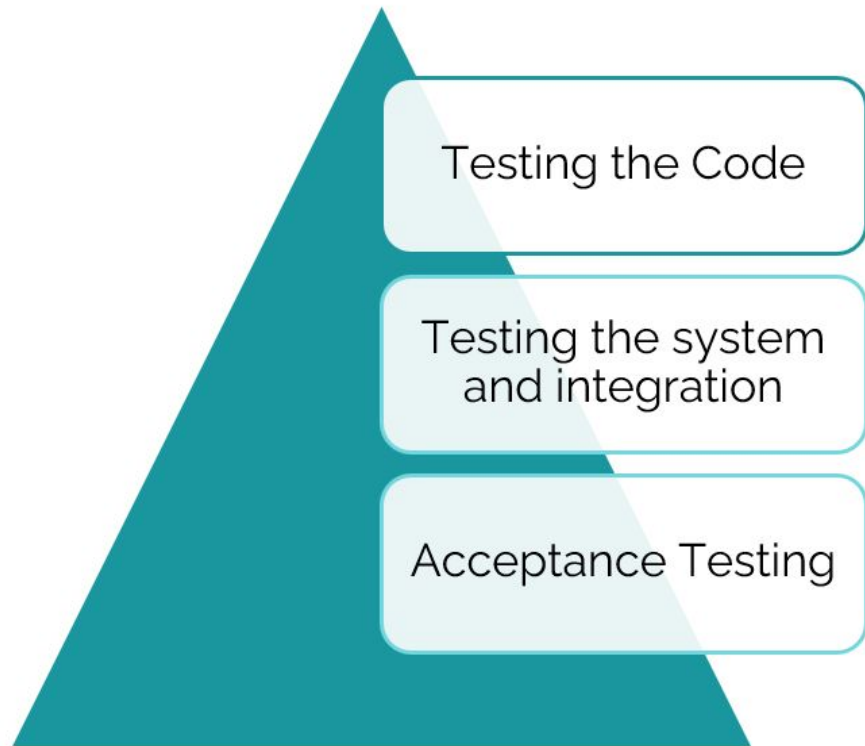


Testing Principles



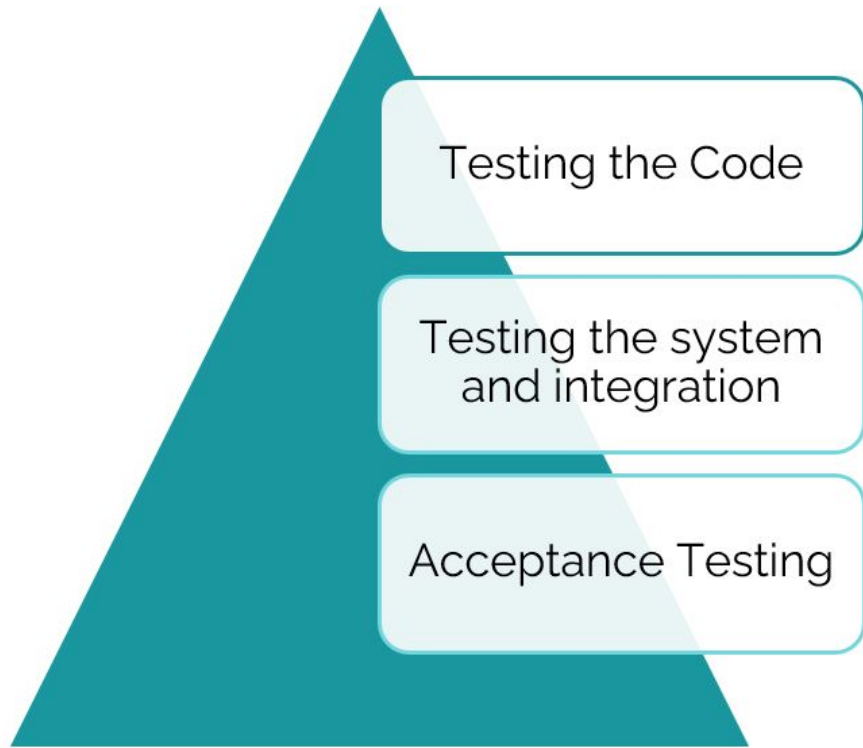
Regardless of approach, testing needs to be carried out at different levels and will need overall management.

- The code itself is usually tested by developers.
- Some developers create tests first, then code part of the system and run the code, others code first then test. Either way, it is usual for developers to test each string of code as they go along. This way they can check one thing works, before building on it.
- Developers then test the modules they are building on completion.



Testing Principles

- This type of testing is known as **component testing**.
- Sometimes the code is checked (scrutinised for errors), this is known as an inspection and may be manual or automated. This is also known as **static testing** (because it does not involve running the code).
- When tests are carried out by running the code and checking that the expected outcomes are achieved, this is known as **dynamic testing**.



Other Types of Testing

Integration

- The interfaces and data transfer between applications

Regression Testing

- Testing what has already been tested, once it has been built upon further – to ensure what was originally built has not regressed when new code is added.

Testing Non functional elements

- Such as capacity, load/stress, testing different environmental aspects.



Specific testing tools (purpose built testing software) is commonly used to test all of the above.

Non-Functional Testing

Testing teams usually use specific software tools to help with integration testing and with tests for non-functional aspects such as performance tests, load tests, stress testing, and demolition testing. Usability aspects are usually carried out as part of UAT.



Acceptance Testing

Acceptance Testing - Often known as **User Acceptance Testing (UAT)** should be carried out by end users of the system or if the end users are external, by internal representatives.

Sometimes where software is built for external clients, a specific type of acceptance testing is carried out known as **Alpha** testing if carried out on the developers site and **Beta** testing if carried out by the beneficiaries on their external sites.

User acceptance testing is black box in nature (users would not be expected to understand the code). It is system testing, but in the wider business system context. It may include manual process elements and connection to real world hardware and peripherals.

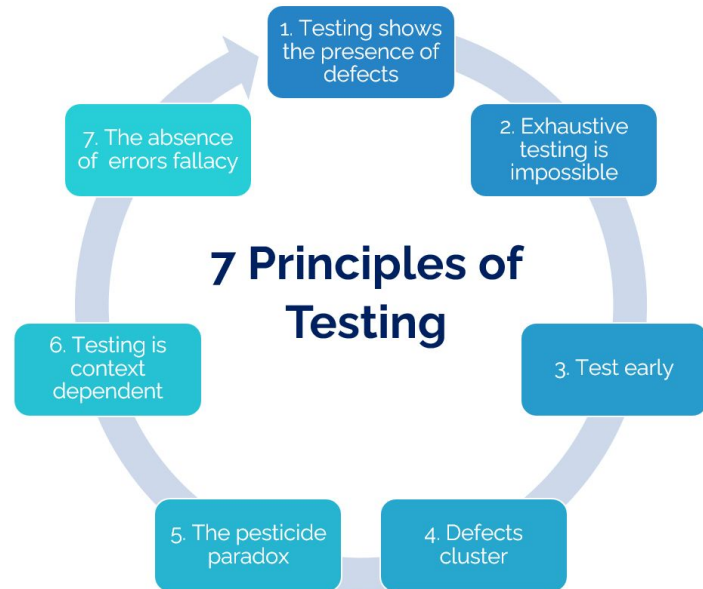


7 Principles of Testing

The principles of testing are applicable regardless of approach.

1. Testing shows the presence of defects

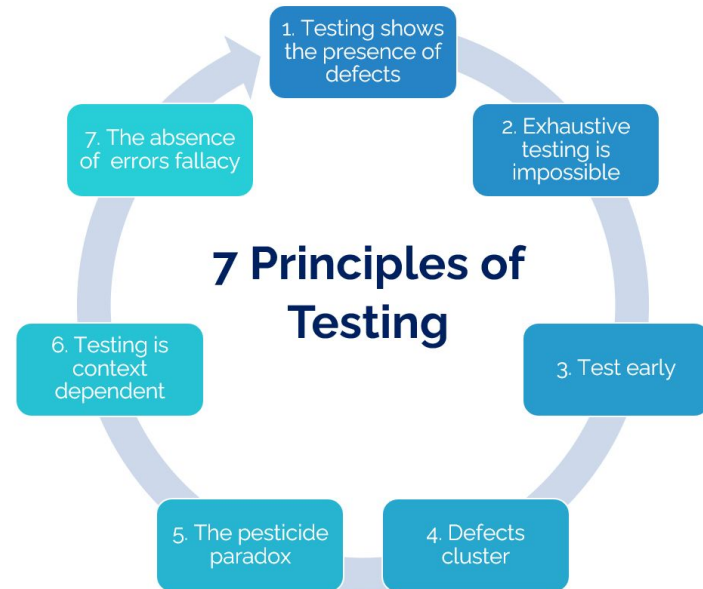
- Thorough testing can reveal defects and increase the probability of their being no or very few defects left undiscovered
- It is virtually impossible to prove that there are no defects
- Therefore testing shows the presence of defects, not their absence



7 Principles of Testing

2. Exhaustive Testing is Impossible

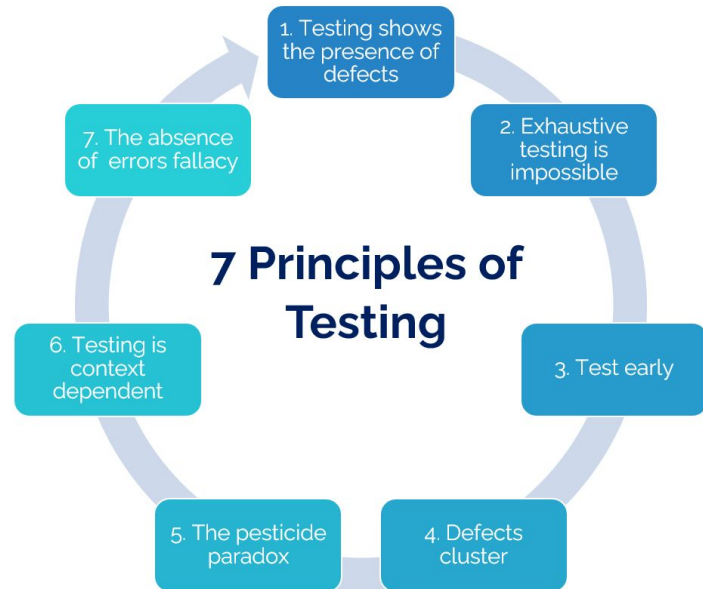
- To test absolutely exhaustively would be to find every single combination of scenarios and data combinations. This would be so time consuming, that it would present the risk that the projects cannot finish.
- By the time everything was tested, any defects found and fixed, something else would change.
- Testing mitigates the risk of deploying a system that contains errors, but attempting to test exhaustively presents a real risk that nothing will ever be fully tested and ready to implement.



7 Principles of Testing

3. Test Early

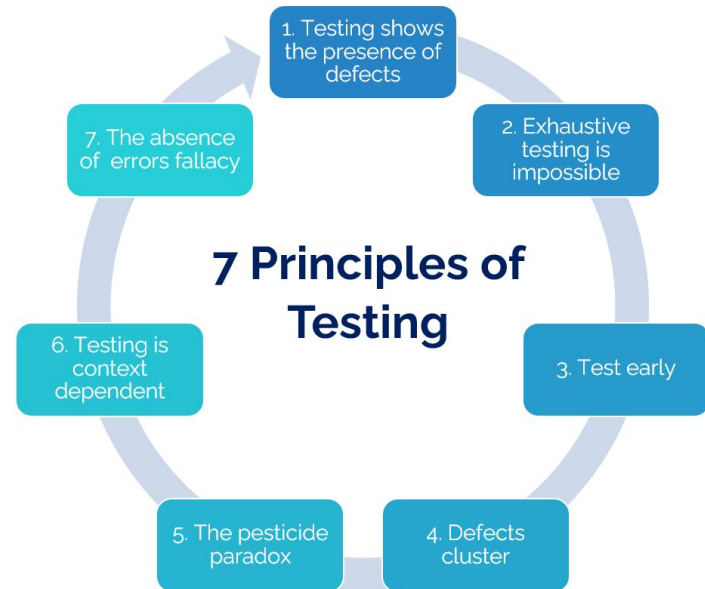
- The earlier testing is carried out, the more cost effective it is.
- When errors and defects remain undiscovered, the error compounds (incorrect requirements, lead to incorrect code, lead to incorrect system).
- Testing earlier (even if it is a static review of a requirements or design specification) is likely to reduce the risk of compounding the error, this saving time and money.



7 Principles of Testing

4. Defects Cluster

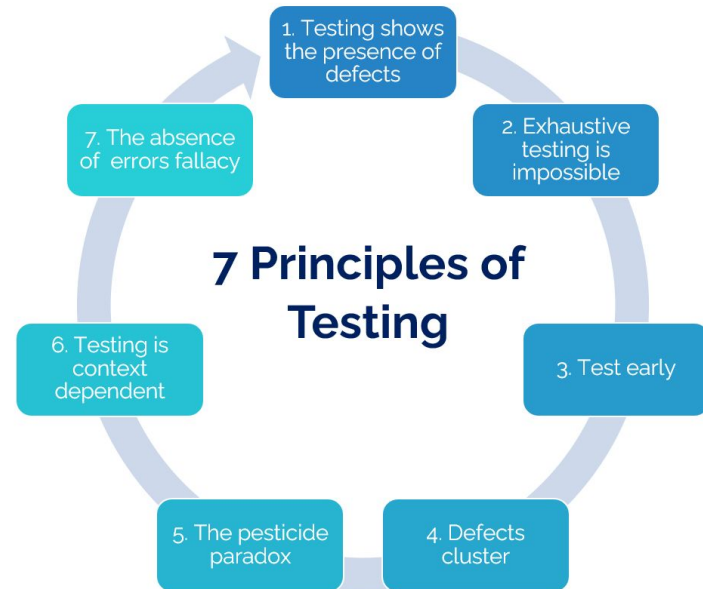
- Defects tend to cluster together, maybe because one particular developer is working on an area of the code and is less experienced. Maybe an experienced developer had a bad day, maybe a portion of the requirements were not expressed clearly.
- Whatever the reason, the fact is that defect clusters are common. This means that if a defect is found, it is worth concentrating efforts to carry out more testing in the same area, in case a cluster can be revealed and eliminated.



7 Principles of Testing

5. The Pesticide Paradox

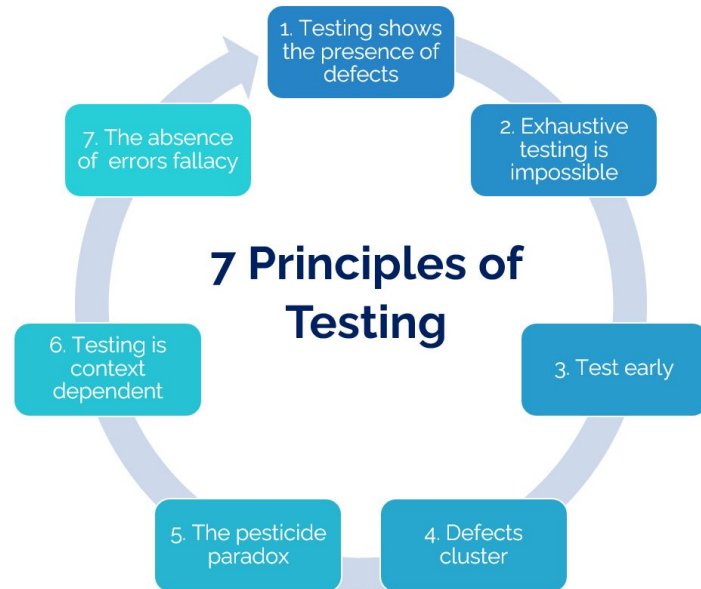
- When testing a system, particularly a large system where re-testing and regression testing is frequent. The code becomes resilient to the test.
- Either consciously or subconsciously, developers are aware of where tests fail and endeavour to ensure the code will pass the tests.
- Whilst this is a good thing, it makes the testing pointless and it is much more fruitful to change the tests to look for different defects.



7 Principles of Testing

6. Testing is Context Dependent

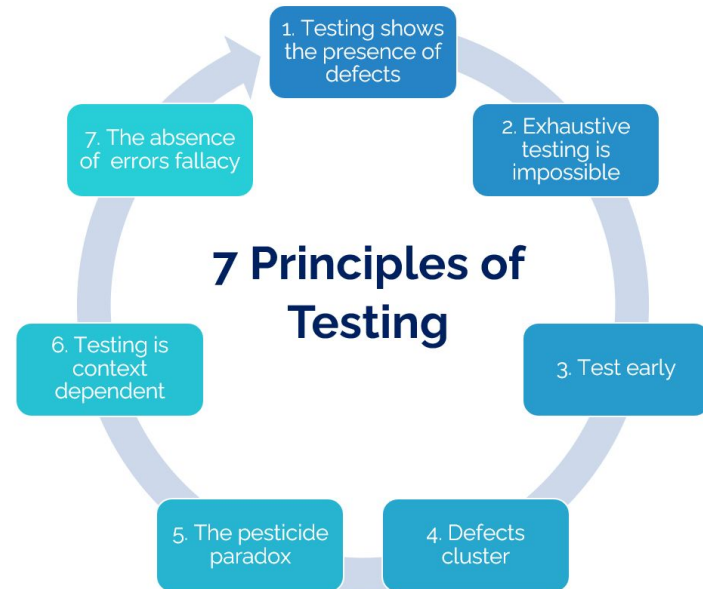
- The test approach for each project should be suitable for the context.
- For example, customer facing systems will need tested in a different way to internal systems.
- The degree of testing is also context dependent. Whether a system is safety critical is a factor, as is the SDLC.



7 Principles of Testing

7. The Absence of Errors Fallacy

- Even if many errors are found and fixed this does not mean that the system is perfect
- It may not be 100% error free.
- It may be imperfect on other ways for example not 100% user friendly
- Nothing can change the fact that the system is likely to be imperfect, but it must be understood that the absence of errors is an unrealistic outcome of testing.

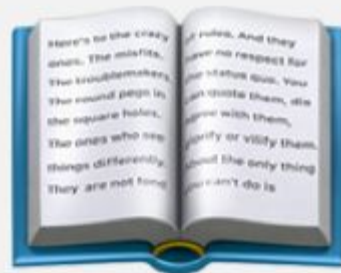


Behaviour Driven Testing

Focuses on testing that the acceptance criteria are met.

This usually takes place at the solution testing level (based on the integrated solution). It focuses on ensuring that the code is correct and behaves as expected by the users. Scenarios form a significant element of the test cases.

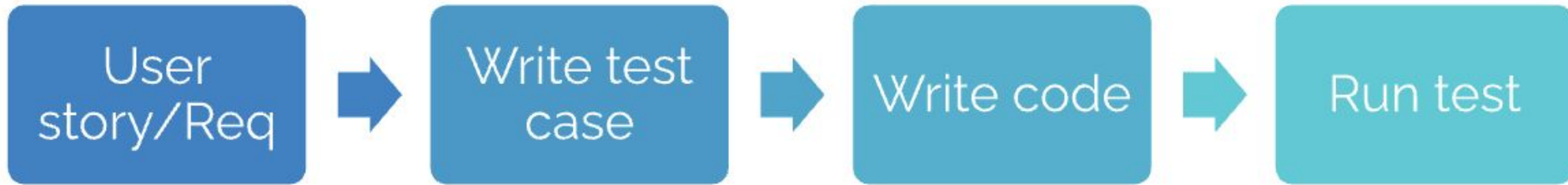
When carried out collaboratively, behaviour driven testing is effective in ensuring that the solution meets the needs of all involved. It can ensure that software designers, testers, developers and operational maintenance staff are happy with the behaviour of the system as well as business end users, internal front end system users, external front end system users and Management Information (MI) and reporting aspects.



Test Driven Testing



This is usually carried out at the component test level



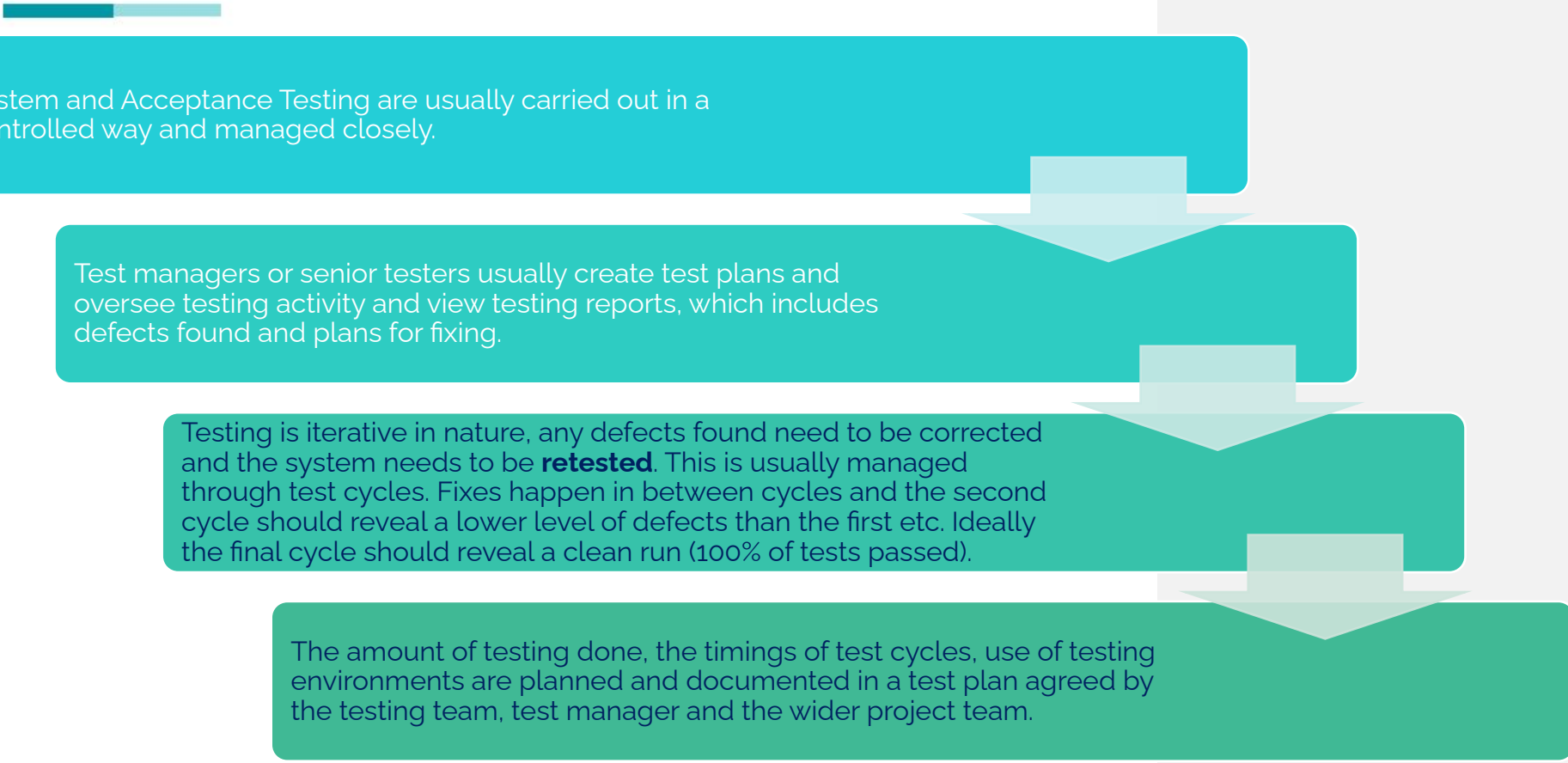
If the code fails the test, the code is changed and the test rerun etc. Test driven development tends to speed up the development and component testing processes and allows focus to remain on writing the code. Giving the software developer the confidence that one string of code is correct, before moving on to the next.

Waterfall Testing

Testing is a distinct stage of the waterfall lifecycle. Some elements may be tested within the development stage, but testing is carried out at all levels post development. Test Management is highly controlled.



Waterfall Testing

The diagram illustrates the Waterfall Testing process as a series of four descending steps. Each step is represented by a teal-colored box with a white arrow pointing downwards to the next box. The boxes are arranged in a staircase pattern, starting from the top left and moving towards the bottom right. The first box is the largest and is a lighter shade of teal, while the subsequent boxes are progressively smaller and darker shades of teal. The text within each box describes a key aspect of the testing process.

System and Acceptance Testing are usually carried out in a controlled way and managed closely.

Test managers or senior testers usually create test plans and oversee testing activity and view testing reports, which includes defects found and plans for fixing.

Testing is iterative in nature, any defects found need to be corrected and the system needs to be **retested**. This is usually managed through test cycles. Fixes happen in between cycles and the second cycle should reveal a lower level of defects than the first etc. Ideally the final cycle should reveal a clean run (100% of tests passed).

The amount of testing done, the timings of test cycles, use of testing environments are planned and documented in a test plan agreed by the testing team, test manager and the wider project team.

The BA Role in Testing

One thing that we can be sure of is that all organisations are different

- They all define the BA slightly differently
- Responsibilities and expectations differ
- The boundaries between roles differ

So what does good practice look like?

- When it comes to testing, there should be a degree of independence between the doer and the tester. This means that some of the testing should be carried out by people who haven't been involved in the creation of the requirements, design or build.
- This also means that BAs shouldn't carry out functional/system testing or UAT, but they can support with both.



The BA Role in Testing

BAs can and probably should help with identification of test scenarios (which helps to establish what will be tested).

A BA can help with the creation of **test scripts**, by helping to establish what use cases within the system need to be tested and establishing what data will be needed to create the tests. BAs can work with Subject Matter Experts (SMEs) to source the data and the best real life* examples to use in test.

*Note that real life examples may undergo data changes in order to de-personalise customer data for data protection purposes.



BA Support for User Acceptance Testing

- UAT should be carried out by end users where possible. For internal applications or internal use of external applications, this means that the testing will be carried out by the people who are expert users of the new system and are probably expert users in the old equivalent system if there is one. These users would ideally have been involved in other aspects of the project such as requirements engineering and process design.
- BAs can liaise between the business users and IT developers in ensuring that test environments, log ins and data uploads are ready for UAT. They can discuss UAT cases with the users and sit with them when carrying out the first tests. BAs may also help users to record the test results and feed them back to the project. UAT environments may need to be refreshed with new data, or fast forwarded to trigger any time triggered events that need to be tested. The UAT will include front end use of the system and possibly some manual processing aspects.



BA Support for Defect Management

When tests fail (when the actual result is different from the expected result), investigation as to the root cause of the defect needs to be carried out.

Firstly the tester who has identified the defect will check the environment and the test case, if everything looks ok, he/she will log the defect (usually in a defect management software tool) and flag it for investigation.

Investigation is carried out by the developers - to check the code

Developers or architects - to check the interfaces

Business Analysts - to check the requirement and the test script



BA Support for Defect Management

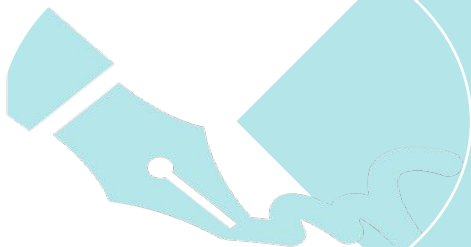
Regular meetings take place during the testing stages to ensure that defects are being investigated and fixed. The BA needs to be involved in these discussions, partly to report on their findings, but also to consider what to do if a defect cannot be fixed in time. The BA can then talk to the team regarding timelines and the Business regarding workarounds.



BA Sign-Off



The BA, or possibly just a senior BA on a project may be asked to sign-off testing. They will not be the only signatory. If they are not asked to sign-off officially, they may be asked to contribute to the decision as to whether enough testing has been done and the project or incremental deployment can proceed to go-live.



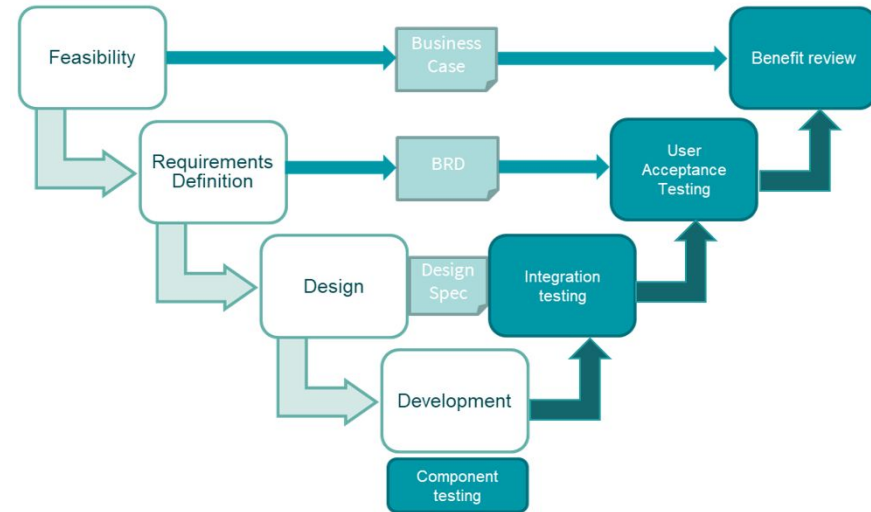
BAs support testing, rather than carrying out the actual tests, Sign-off should be limited to stating that the test have to the best of your knowledge been carried out in the specified environments and including your opinion as to whether the tests have been carried out according to the test plan. BA may also be asked to sign off the test plan/approach itself.



Testing When Using the V Model

The V model is based on the principle that testing should be carried out early. In fact, the test plan may be one of the earliest deliverables following the terms of reference.

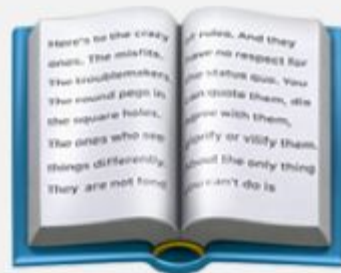
In short, all of the testing that would have been carried out during a waterfall lifecycle will still be carried out. Initially, the test plan, business case, business requirements document, design specification and the code will undergo a static review at the end of the stages on the left hand side of the model. The code and interfaces will then undergo dynamic testing as per the usual waterfall approach, as will the UAT.



Agile Testing

When taking an Agile approach, as a minimum, system testing is carried out for each sprint and integration testing and regression testing for each increment.

Regression testing is important for all approaches, but is carried out more frequently in the agile environment. Regression testing is when code that has previously passed tests, is tested again after adding more functionality, to ensure the addition of the new hasn't caused a malfunction in the old. BAs may need to support, or may need to explain it to business users.



Agile Testing

Regression testing is particularly important in **agile approaches** because there are multiple deployments. Every time new functionality is deployed, the existing build should be regression tested to ensure that the newly released software will not unexpectedly change the functionality or non-functional aspects of the live software. This may be carried out in the test environments, using a copy of the existing environment along with the new code.

Component, integration and UAT are carried out for each software release. Some component testing and possibly integration testing may also be carried out during each sprint. As with waterfall projects, exit criteria need to be defined so that they know the extent to which they need to test. Defect management is also carried out in the same way, but all of these activities will be carried out more frequently.



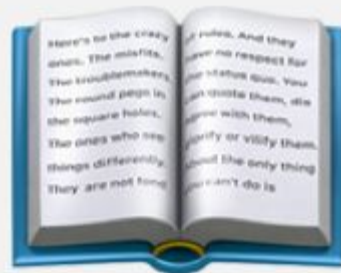
Refactoring

Sometimes code gets messy and inefficient and there is a greater risk of this when taking the Agile approach.

Refactoring is the process of reviewing the code with the aim of eliminating mess and waste, leaving simple, tidy and efficient code.

Refactoring the code should not impact the outcome, in other words, the code will achieve the same goals, but in a more efficient and simple way.

Refactoring is often carried out iteratively. The code, or elements of the code may be written and in one iteration and refactored in another and may be refactored again in the future as more code is added. Regression tests should follow refactoring exercises to ensure that the software still behaves as expected.



Emergent Design



When using the Agile approach, there may be a thought to overall design based on the initial backlog, but emergent design is more likely.

Upfront Design

- Can constrain the solution
- This is not encouraged in Agile methodologies, however a high level view of the overall design can be made after the initial backlog is produced

Last Responsible Moment

- Design decisions should be made at the last responsible moment
- This allows teams to make decisions based on evidence (empirically)

Opportunistic Design

- Making design decisions as they arise, at the most opportunistic time to make a difference, and confidently make the right decisions

Continuous Integration / Continuous Deployment (CI/CD)

Continuous Integration

- As code and components are developed, they need to be integrated.
- They may be integrated so that they can be delivered as an individual component coupled to the existing software

Continuous Deployment

- New code or components are deployed into the live environment throughout the project or over a period of time.
- Each deployment may include code produced in one or many sprints

Value

- Software releases should add value
- If not, they should be released with further code at a later date – when they do add value



Continuous Integration / Continuous Deployment (CI/CD)

Continuous integration and deployment can be problematic. A piece of code may work well in isolation, but when integrated it could break, or it may break existing code that it is integrated with, hence regressing the build.

This gives rise to the need for continuous component testing, integration testing and regression testing.

Testing is often intense and time consuming and can slow the process down. Therefore automated tests are often built and specialist integration software testing tools are used to speed up the process. Tests can then run straight after each element of a build/integration and any defects can be fixed straight away.

